

Recurrent Neural Networks

Dr. Sunil Kumar Vuppala

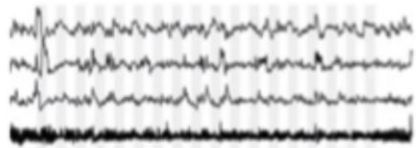
www.linkedin.com/in/sunilvuppala/

Agenda

- Sequence modeling - what? why? Where?
- RNN - How to do sequence modeling?
- LSTM
- Applications

Sequence modeling

"This morning I took the dog for a walk."



to model sequences, we need:

1. to deal with **variable-length** sequences
2. to maintain **sequence order**
3. to keep track of **long-term dependencies**
4. to **share parameters** across the sequence

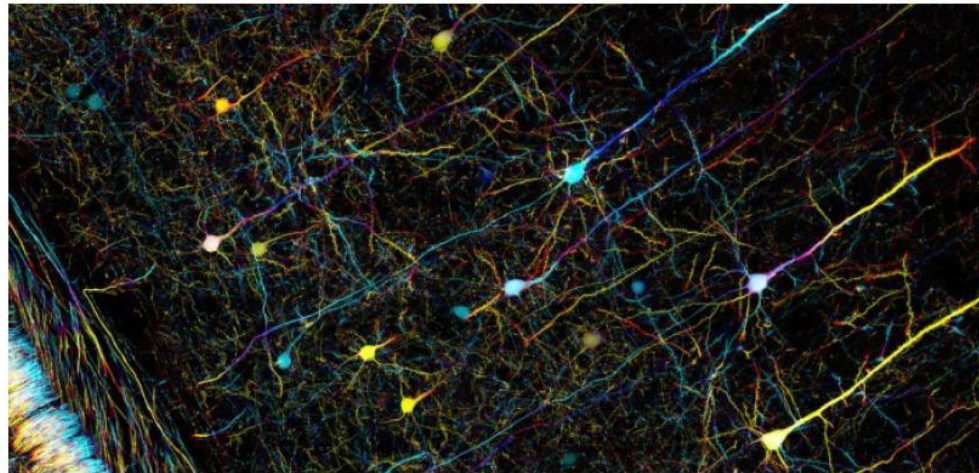
Let us start with applications

- Stock prediction
 - Predicting which stocks might go up or down in future based on the past data
- Chatbots:
 - Understand the context
- Speech recognition
- Music composition
- Protein and DNA sequences



Inspiration from biology

Are all the connections in brain feed-forward?



Are there feedback connections?

Language Modeling

Task of predicting the
next word

(given earlier words)

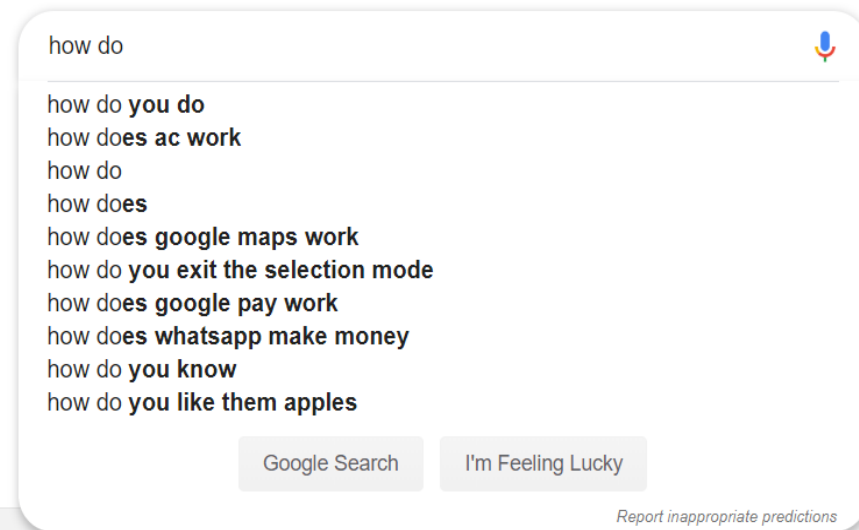
Language Modeling (in Math)

Given a sequence of words $x^1, x^2, x^3, \dots, x^t$,
find the probability distribution for the next word

OR

$$P(x^{t+1} = w_i \mid x^1, x^2, x^3, \dots, x^t)$$

Where w_i is a word in the vocabulary



A Language Model at Work

A solid orange horizontal bar spans the width of the slide at the bottom.

N-gram Language Models

- What is n-gram?
 - n-gram is a chunk on n-consecutive words
 - **Uni**grams : “the”, “rain”, “just”, “stopped”, “and”, “the”, “sky”, “is”
 - **Bi**grams : “the rain”, “rain just”, “just stopped”, “stopped and”, ..., “sky is”
 - **Tri**grams : “the rain just”, “rain just stopped”, ..., “the sky is”
 - **4-**grams : “the rain just stopped”, “rain just stopped and”, ..., “and the sky is”
 - Idea: Collect statistics about how frequent different n-grams are and use these to predict next word

N-gram Language Models ...

1. How does n-gram model work?

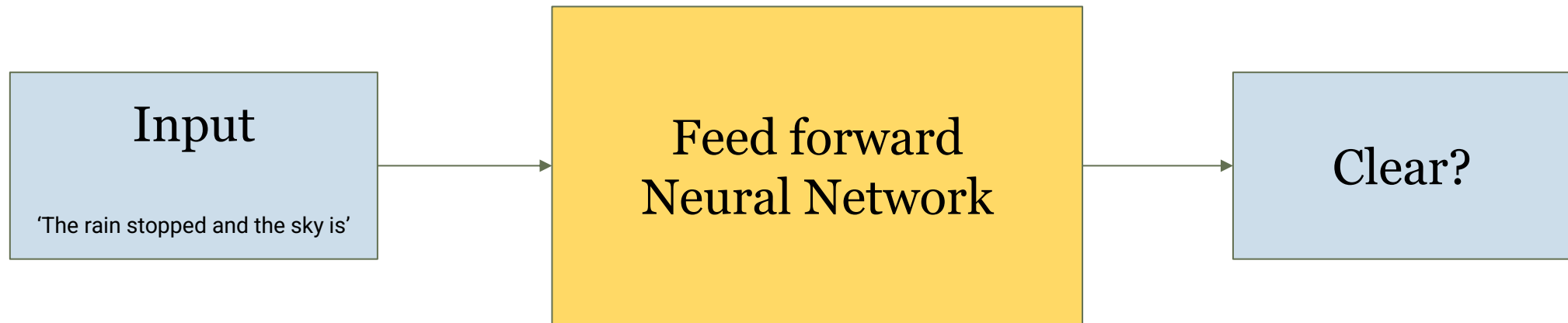
1. Assume that next word is dependent on preceding n-1 words
 1. e.g Next word is dependent on last 3 words “the sky is”
2. From a large corpus, collect frequency metrics for n-1 grams
 1. e.g How often “the sky is” appears in the corpus (simple count)
3. Collect frequency of n-grams which have first n-1 words same as previous step
 1. e.g How often n-grams like “the sky is **blue**” or “the sky is **dark**” or “the sky is **clear**” appear in the corpus
4. Divide different n-grams frequencies (step 1.3) with n-1 gram frequency (step 1.2) to get next word probability.
 1. In the corpuse, “the sky is” appears **100** times
 2. “the sky is blue” **47** times, “the sky is dark” **33** times and “the sky is clear” appears **20** times
 3. Based on probability, the word will be **blue** in this case (47% probability)

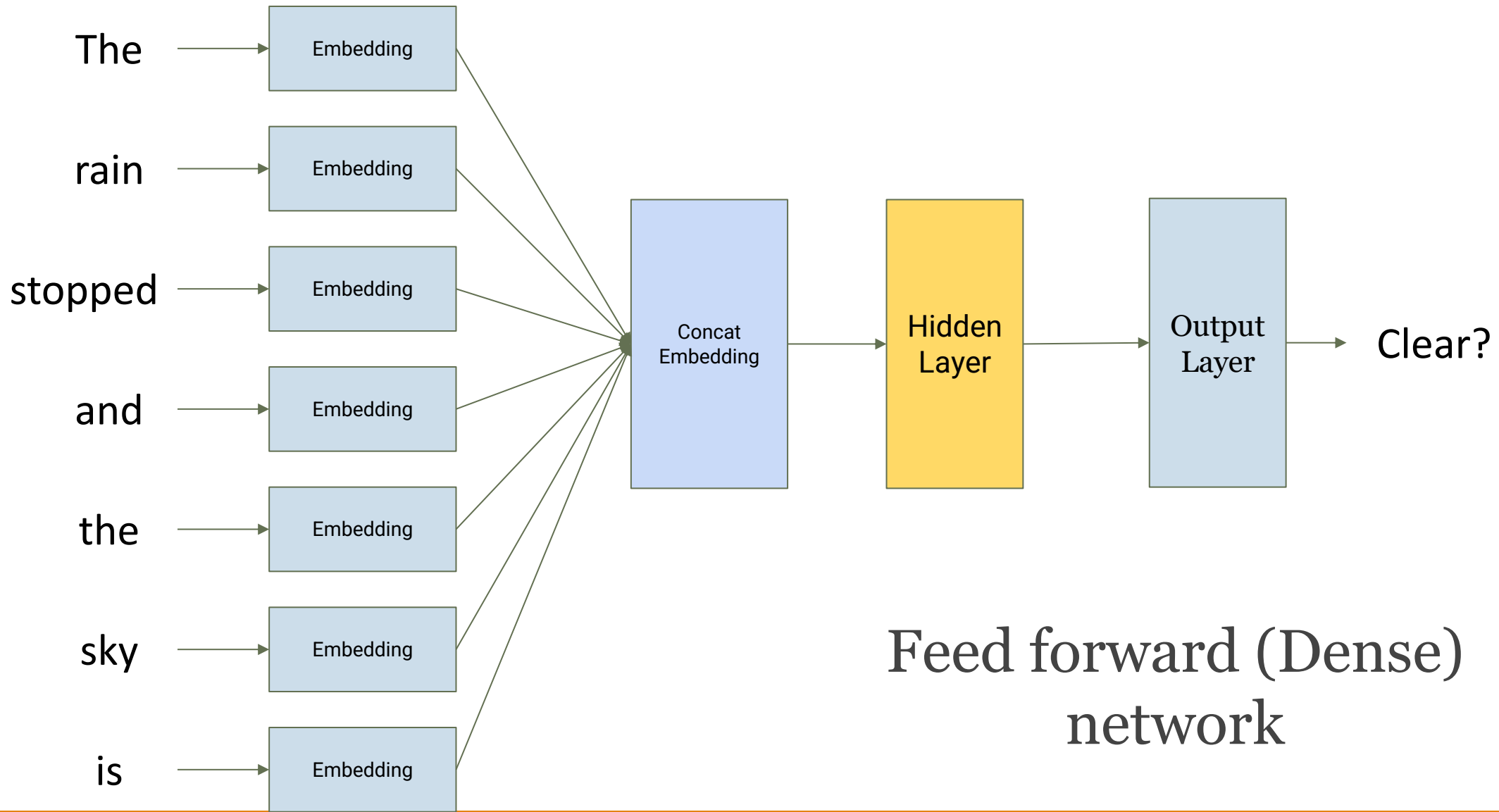
Issues with N-gram Language Models

1. Need to **store** count of all possible n-grams *i.e* model size is huge
2. Increasing 'n' can make **model size very very big**
3. **Sparsity Problem**
 - a. What if “the sky is” never occurred in corpus
 - i. We can then use 'sky is' as a potential solution in this case (called backoff)
 - b. The problem becomes worse when we increase 'n'



Feed forward (Dense) network





Feed forward (Dense) network

Advantages (over n-gram model)

- No sparsity problem
- Model size is much smaller (no need to keep n-grams and their counts)

but...

- Window (number of input words) size has to be **fixed**
- Large window can result in **huge model size**

We need a Neural Architecture which can process any length input

Recurrent Networks

Feed forward networks:

- Information only flows one way
- One input pattern produces one output
- No sense of time (or memory of previous state)

Recurrency

- Nodes connect back to other nodes or themselves
- Information flow is multidirectional
- Sense of time and memory of previous state(s)

Possible applications of RNN's are in domains where data is **sequential**.

For example:

Speech and Text (NLP), Music

Protein and DNA sequences

Time series from trade data

Conceptual understanding of RNN

Training data of RNN needs multiple sequences of data

How this text can be input to a neural net?

- *I wish it had longer battery life.*
- *The phone has adequate processing, but does not have a premium touch and feel.*
- *The processor is fast, but the look and feel is low end.*
- *An affordable phone with lot of power.*

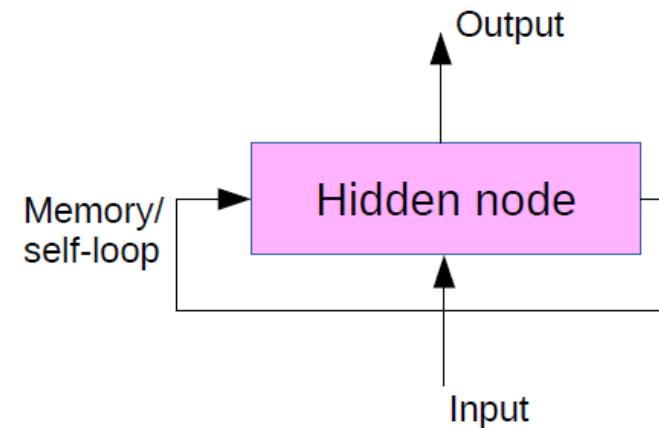
One option is Word embedding such as word2vec

Conceptual understanding of RNN

Training data of RNN needs multiple sequences of data

Input No
1: I wish it had longer battery life
2: I wish it had longer battery life
3: I wish it had longer battery life

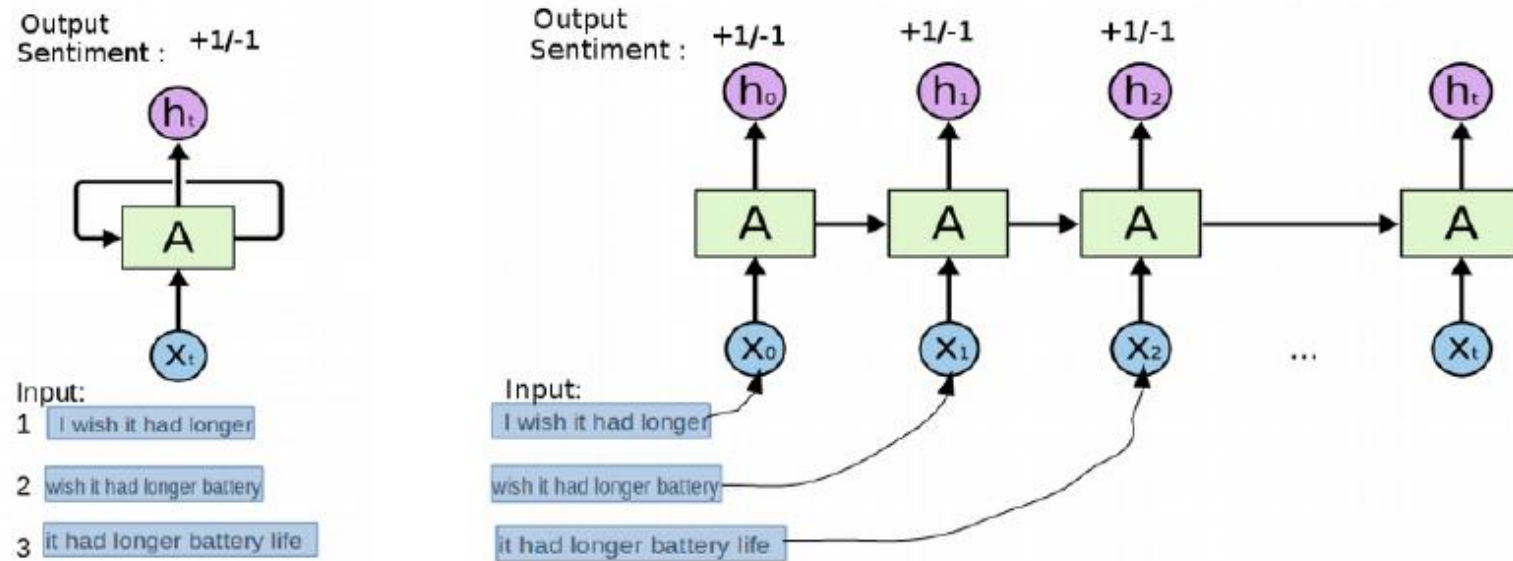
Ex: You can input five words at a time using a sliding window



RNN nodes have self-loops. This ensures that some representation of the previous data also contributes to the output

Conceptual understanding of RNN

Example input: I wish it had longer battery life

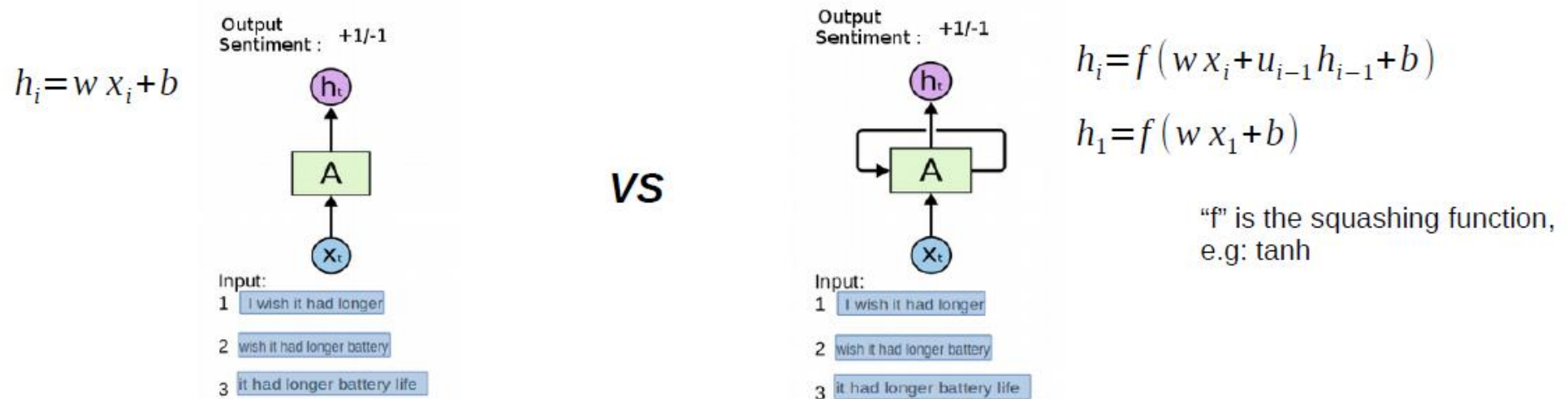


When sequential inputs are present, the network in effect turns into a large number of hidden nodes with the previous output being propagated

Conceptual understanding of RNN

The self loop gives the network a memory of previous states

What will happen if the previous state was not present?



E.x.: Consider input no 3. The feed-forward NN will mostly output +1 because input 3 has no negative word. The recurrent neural net will however use part of the previous output, so it may give a value less than +1

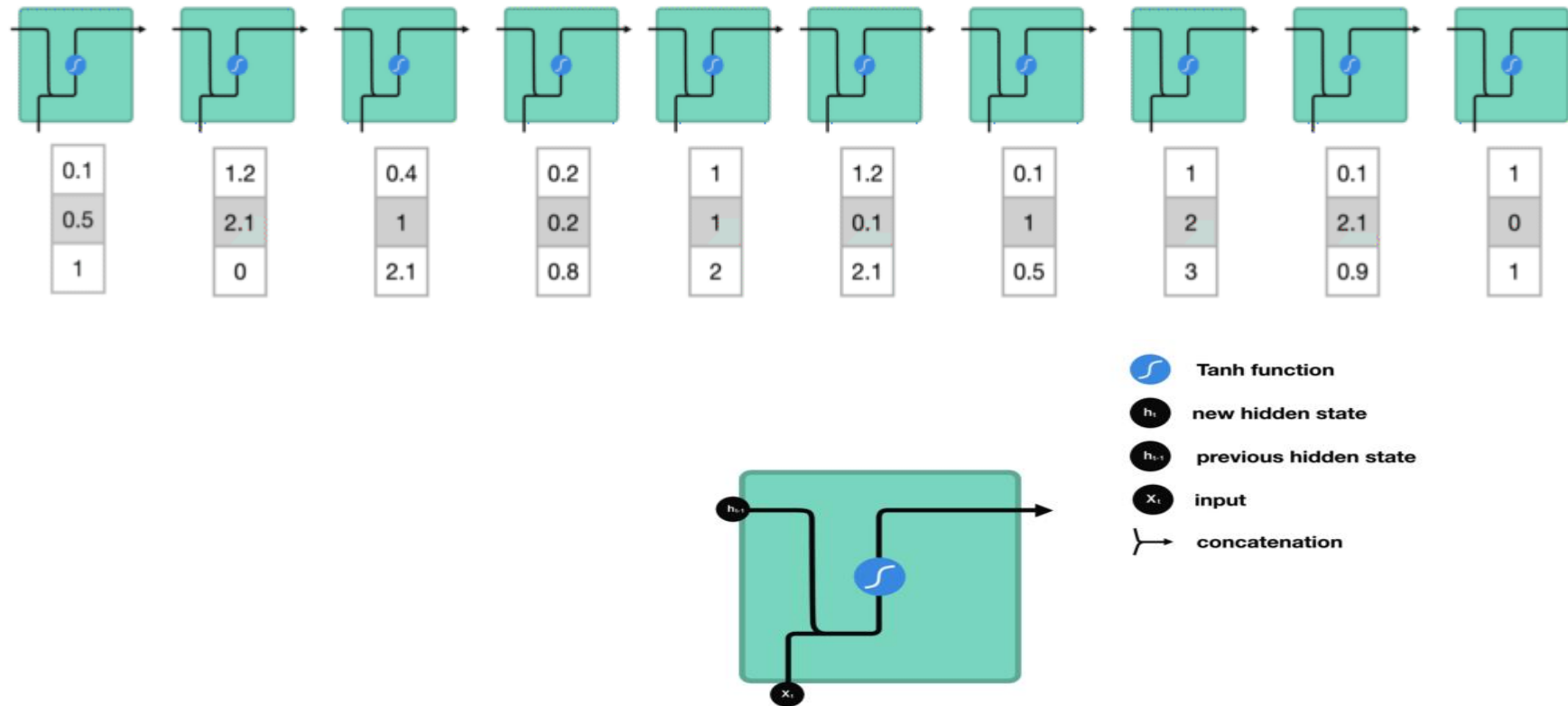
– Effectively, the output in a feed-forward could be: -1, -1, +1. But the RNN could give: -1, -1, 0.25

Sequence modeling requirements

To model sequences, we need:

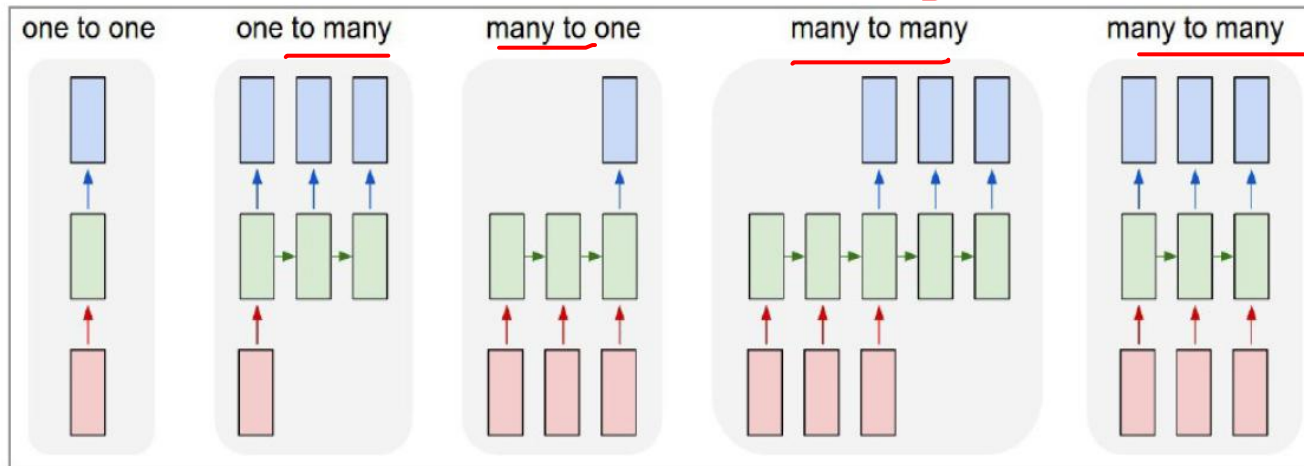
1. To deal with VARIABLE LENGTH sequences
2. To maintain SEQUENCE ORDER
3. To keep track of LONG TERM DEPENDENCIES
4. To SHARE PARAMETERS across the sequence

RNN flow



<https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>

Different Networks



Input are in red, output vectors are in blue and green vectors hold the RNN's state. From left to right

(1) **Vanilla mode of processing without RNN**, from fixed-sized input to fixed-sized output (e.g. image classification).

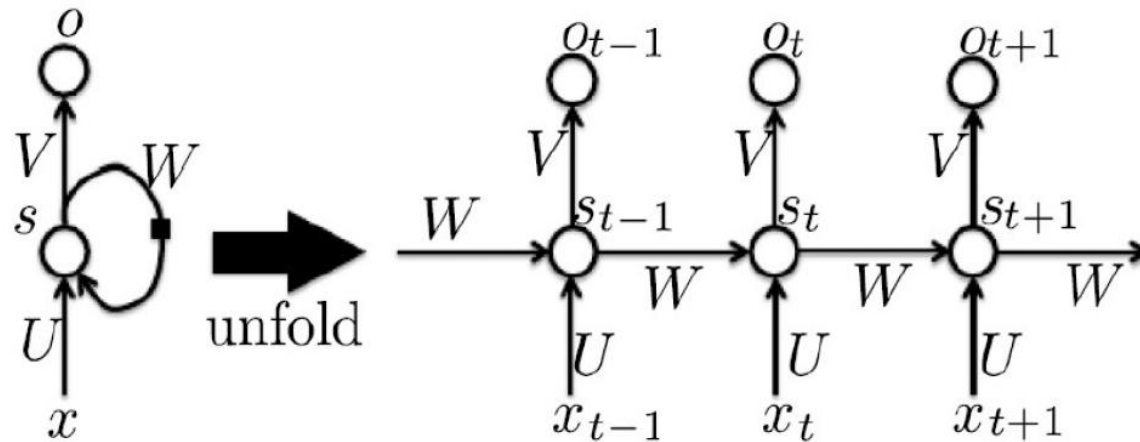
(2) **Sequence output** (e.g. image captioning takes an image and outputs a sentence of words).

(3) **Sequence input** (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment).

(4) **Sequence input and sequence output** (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French).

(5) **Synced sequence input and output** (e.g. video classification where we wish to label each frame of the video).

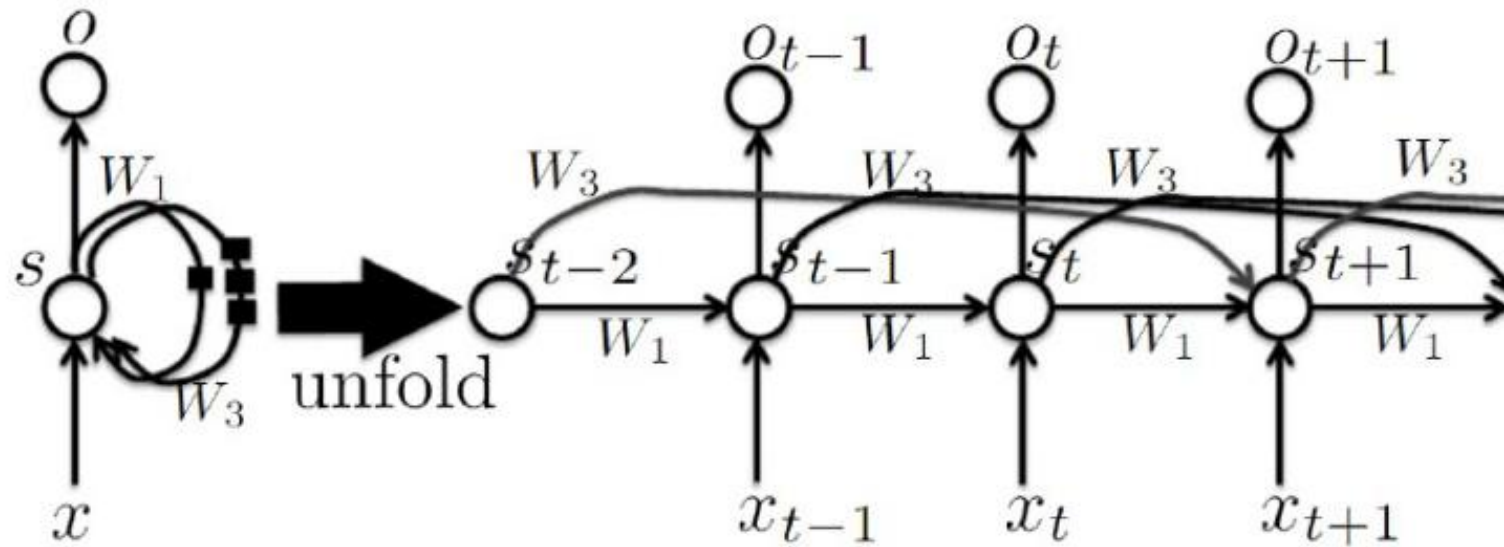
Back Propagation Through Time (BPTT)



- BPTT begins by unfolding a recurrent neural network through time .
- Training then proceeds in a manner similar to training a feedforward neural network with **backpropagation**.
- After each pattern is presented, *weight updates are computed for time instance*. All weight updates are averaged together so that they all have the same weights.

Long delay RNN

Use recurrent connections with long delays.



Issues in RNN

- Vanishing Gradient Problem / Exploding gradients
 - Partial derivatives may be very large or small
- Using BPTT results in a **vanishing gradient** problem.
- Final output is a composition of many non-linear transformations (sigmoid/tanh)
- Error is propagated back in time where each time step is exactly equivalent to propagating through an additional layer of a feed forward network.

How much of Past?

- The importance of network state at times which lie far back in the past. **How much of Past?**

Gap between relevant information and where it is needed is important

Predicting the last word in the text:

- “the clouds are in the *sky*”. Not much context is required as obvious. (small gap)
- “I grew up in Karnataka. ... I speak fluent *kannada*”. (large gap)

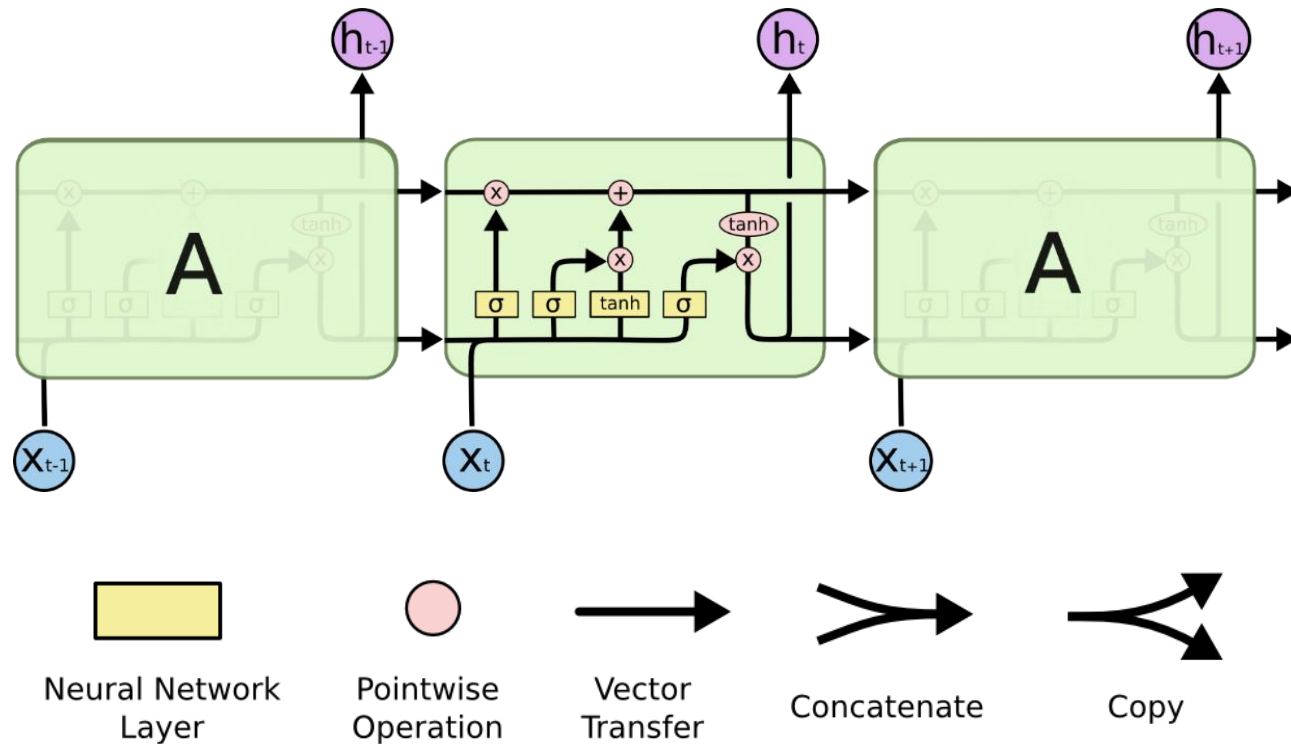
As the gap grows, RNNs unable to learn to connect information

How to solve?

A human carefully analyzes the passage and picks the right features to put in the memory.

- A natural architecture that remembers the relevant past information.
- In an artificial system we need a mechanism which can facilitate ***update of memory only when required.***

LSTM



- **Each line** carries an entire vector, from the output of one node to the inputs of others.
- The **pink circles** are point-wise operations, like vector addition
- The **yellow boxes** are learned neural network layers.
- **Lines merging** denote concatenation
- **Line forking** denote its content being copied and the copies going to different locations.

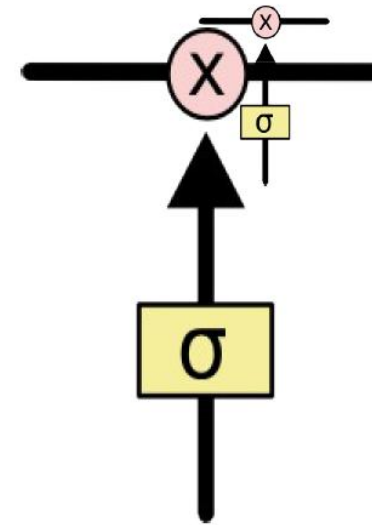
Gates

The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called **gates**.

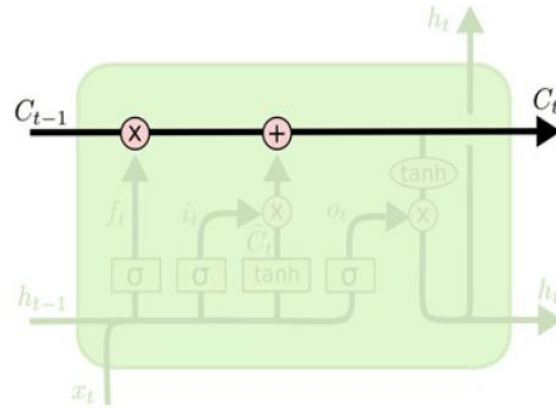
Gates are a way to optionally let information through. They are composed out of a **sigmoid** neural net layer and a **point-wise multiplication** operation.

The sigmoid layer outputs numbers between zero and one, describing how much of each component should be let through. A value of zero means “let nothing through,” while a value of one means “let everything through!”

An LSTM has three of these gates, to protect and control the cell state.



Cell State



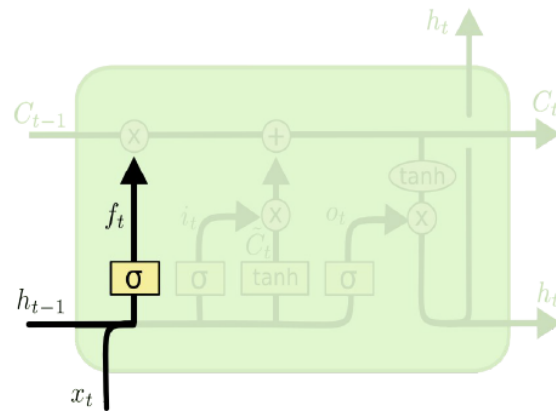
The key to LSTMs is the **cell state**, the horizontal line running through the top of the diagram.

The cell state is kind of like a **conveyor belt**. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.

Forget Gate

The first step in our LSTM is to decide what information we're going to throw away from the cell state. This decision is made by a sigmoid layer called the “**forget gate layer**.” It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} .

1 represents “completely keep this” while a 0 represents “completely get rid of this.”



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

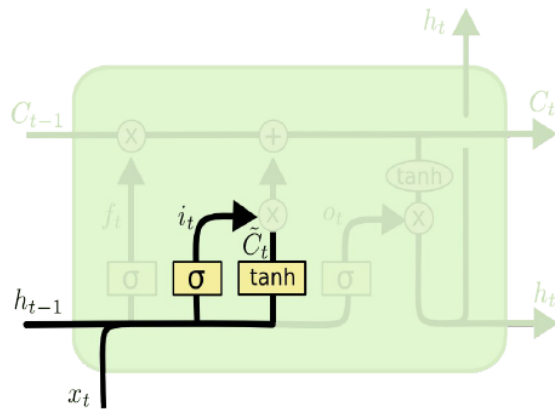
Context and new information

A language model trying to predict the next word based on all the previous ones. In such a problem, the cell state might include the gender of the present subject, so that the correct pronouns can be used.

When we see a new subject, we want to forget the gender of the old subject.

The next step is to decide what **new information** we're going to store in the cell state. This has two parts.

Input gate



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- First, a **sigmoid** layer called the “**input gate layer**” decides which values we’ll update.

- Next, a **tanh** layer creates a vector of new candidate values, C_t , that could be added to the state.

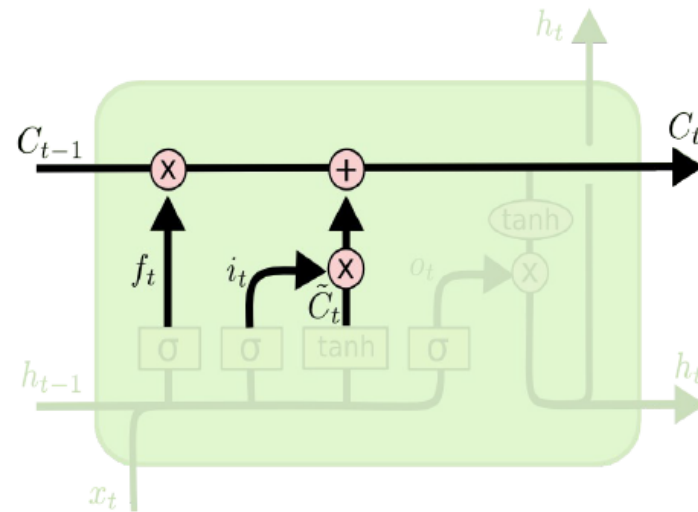
- In the next step, we’ll combine these two to create an update to the state.

In the example of our language model, we’d want to add the *gender of the new subject to the cell state*, to replace the old one we’re forgetting.

LSTM

It's now time to update the old cell state, C_{t-1} , into the new cell state C_t . *The previous steps already decided what to do*, we just need to actually do it.

In the case of the language model, this is where we'd **actually drop the information about the old subject's gender** and add the new information, as we decided in the previous steps.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

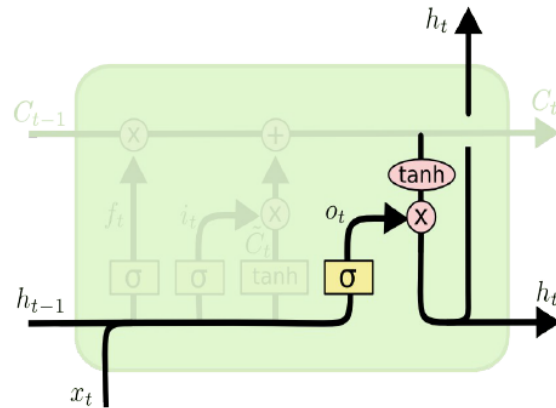
Output gate

Now we need to decide what we're **going to output**.

The output will be based on our cell state, but will be a **filtered** version.

First, we run a sigmoid layer which decides what parts of the **cell state we're to output**.

Then, we put the cell state through **tanh** (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output we decided to.

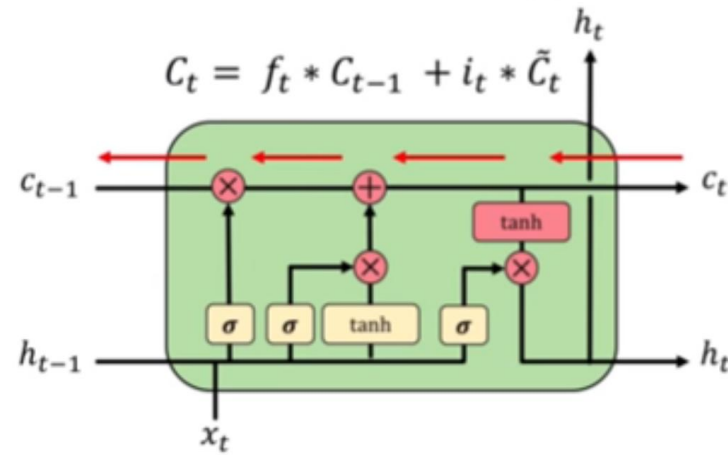


$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

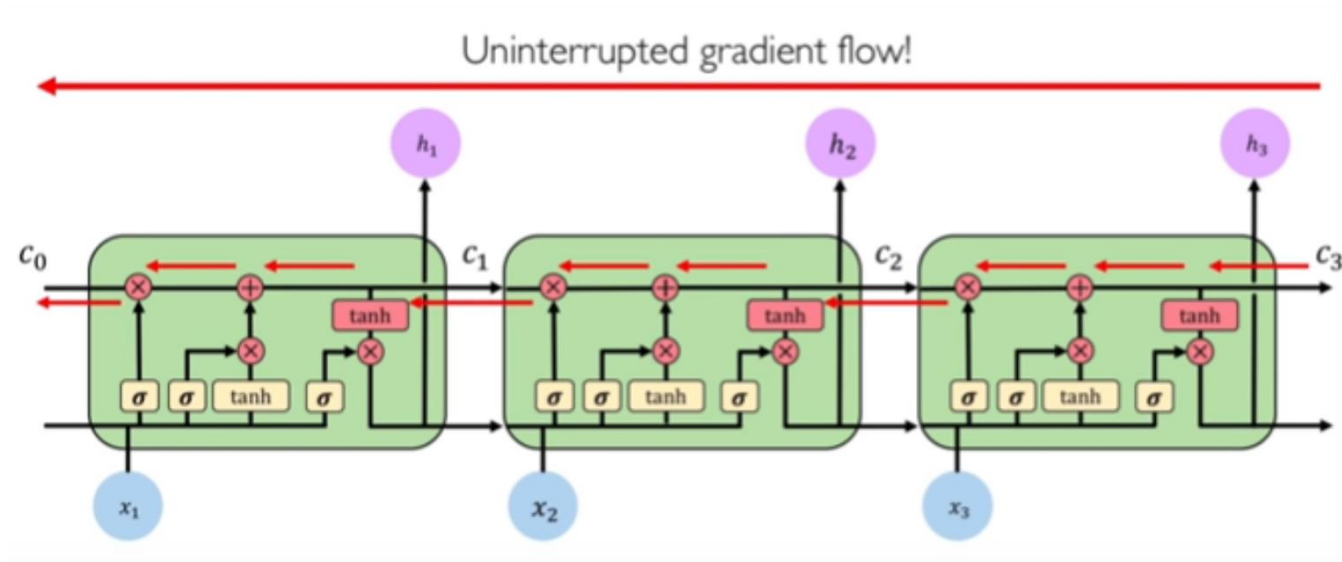
$$h_t = o_t * \tanh(C_t)$$

LSTM gradient flow

Backpropagation from C_t to C_{t-1} requires only elementwise multiplication!
No matrix multiplication $\rightarrow$ avoid vanishing gradient problem.

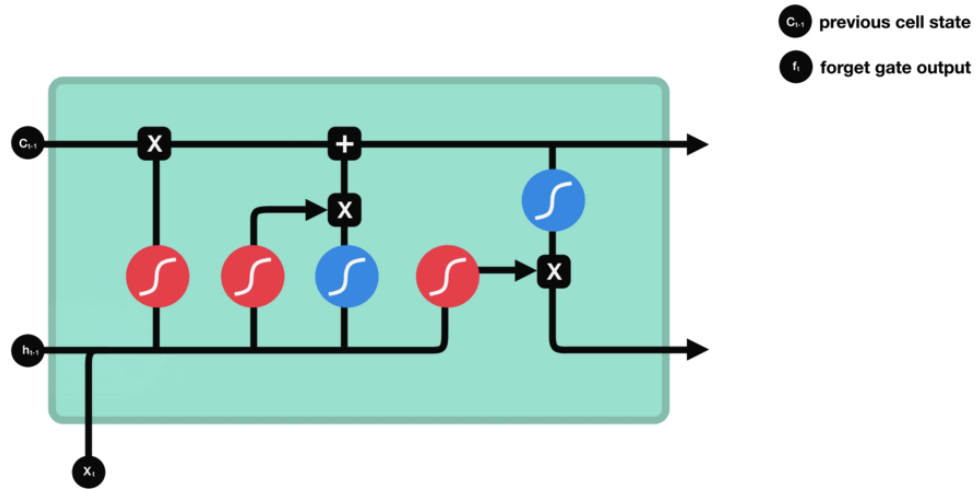


LSTM gradient flow

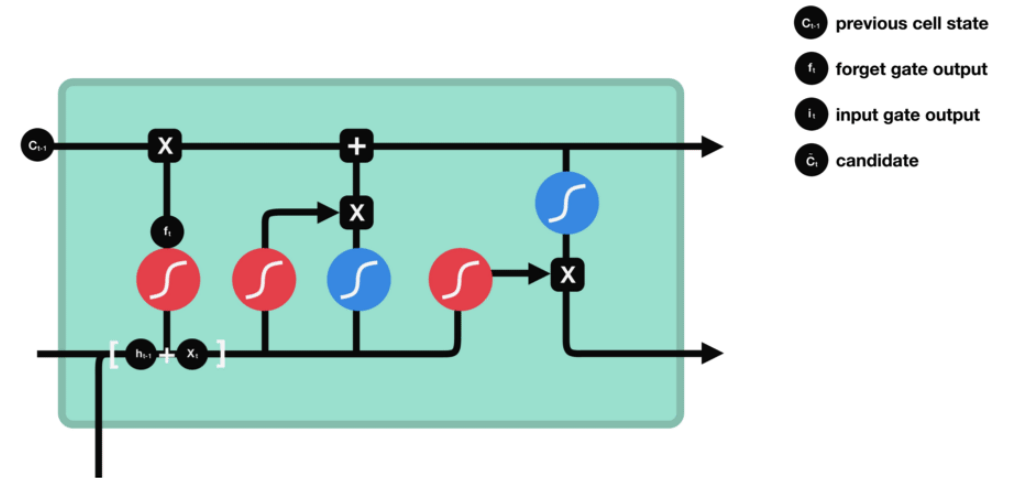


LSTM flow

Forget Gate operations

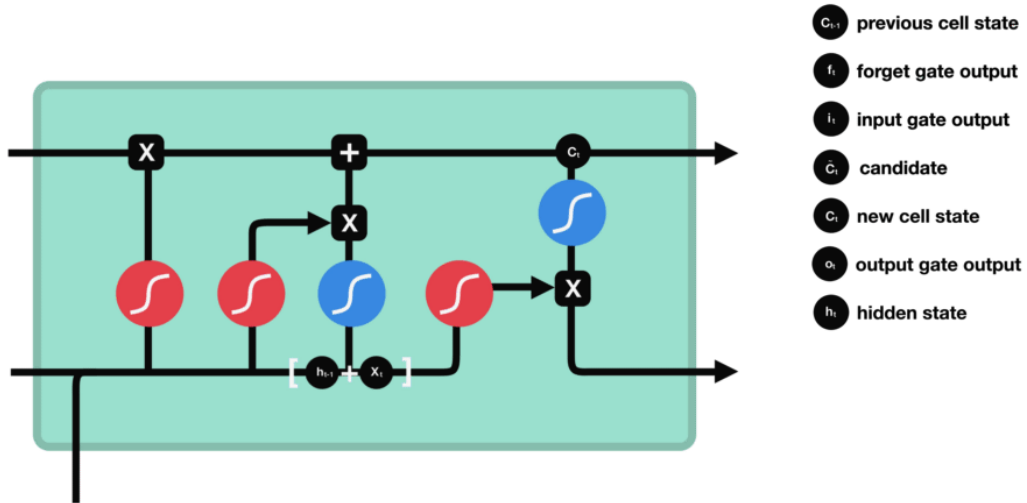


Input Gate Operations

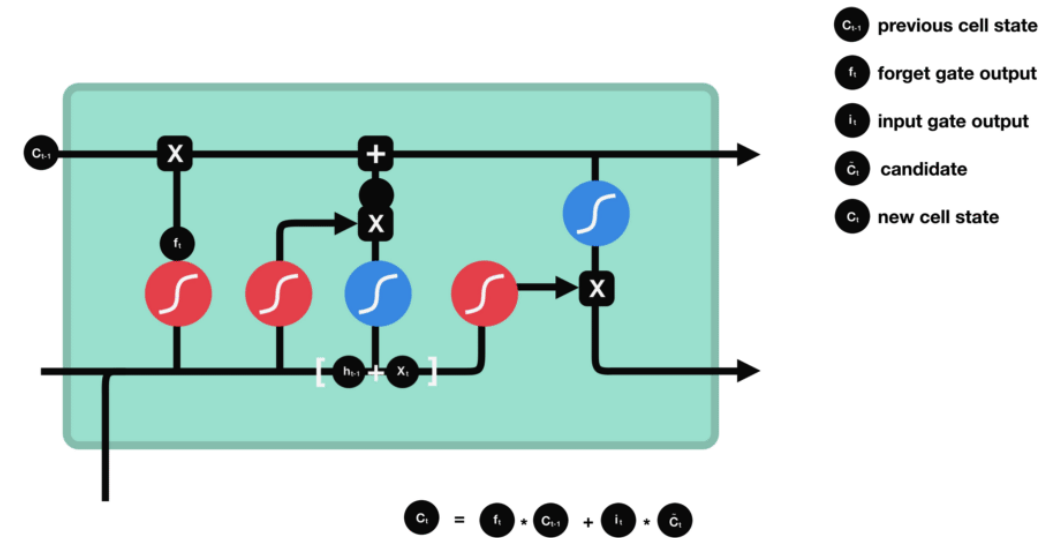


LSTM flow

Output Gate Operations



Cell State operations



LSTM in review

The **Forget gate** decides what is relevant to keep from prior steps.

The **input gate** decides what information is relevant to add from the current step.

The **output gate** determines what the next hidden state should be.

References

<http://introtodeeplearning.com/2017/Sequence%20Modeling.pdf>

<https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>

<http://jalammar.github.io/illustrated-bert/>

<https://arxiv.org/abs/1810.04805>.

<https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

<https://jalammar.github.io/illustrated-transformer/>

<https://machinelearningmastery.com/teacher-forcing-for-recurrent-neural-networks/>

<https://medium.com/analytics-vidhya/a-must-read-nlp-tutorial-on-neural-machine-translation-the-technique-powering-google-translate-c5c8d97d7587>

<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

<https://medium.com/datathings/the-magic-of-lstm-neural-networks-6775e8b540cd>

<https://pub.towardsai.net/chatgpt-how-does-it-work-internally-e0b3e23601a1>